

Лабораторная работа №3

Ознакомление с Агентным подходом к имитационному моделированию

Mehmet Efe Kantoz

14.09.2026

Содержание I

- 1 Информация
- 2 Цель работы
- 3 Задание
- 4 Теоретическое введение
- 5 Выполнение лабораторной работы
- 6 Выводы

Раздел 1

Информация

- Mehmet Efe Kantoz

- Mehmet Efe Kantož
- Студент 3 курс

- Mehmet Efe Kantoz
- Студент 3 курс
- Российский университет дружбы народов им. П. Лумумбы

- Mehmet Efe Kantož
- Студент 3 курс
- Российский университет дружбы народов им. П. Лумумбы
- 1032228428@pfur.ru

Раздел 2

Цель работы

Цель работы

Ознакомление с Агентным подходом к имитационному моделированию и моделью Daisyworld.

Раздел 3

Задание

– Создать рабочий каталог для кода. – Установить необходимые пакеты. – Выполнить предложенный код и преобразовать в литературный стиль. – Сгенерировать из литературного кода чистый код, jupyter notebook и документацию. – Выполнить код из jupyter notebook. – Интегрировать документацию в формате Quarto в отчёт. – Прodelать аналогичные шаги для набора с параметрами. – Результирующие файлы выложить на git.

Раздел 4

Теоретическое введение

Агентный подход к имитационному моделированию (Agent-Based Modeling, ABM) – это метод исследования сложных систем, в котором поведение системы возникает из взаимодействия множества автономных сущностей, называемых агентами. Этот подход особенно полезен, когда поведение системы трудно предсказать из-за нелинейностей, гетерогенности участников или адаптивных стратегий.

Ключевые компоненты агентной модели

Любая агентная модель включает три основных элемента: – **Агенты** – активные сущности со свойствами, правилами поведения и целями. – **Среда** – пространство (дискретное, непрерывное, сетевое), в котором существуют агенты. – **Взаимодействия** – правила, определяющие, как агенты влияют друг на друга и на среду.

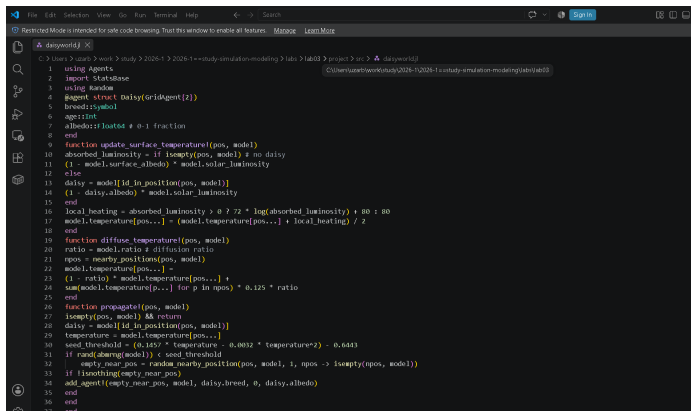
Модель Daisyworld иллюстрирует гипотезу Геи. В мире маргариток произрастают чёрные и белые маргаритки. Их альbedo различается: чёрные маргаритки поглощают тепло, нагревая среду; белые делают наоборот. Маргаритки размножаются только в определённом температурном диапазоне. Взаимодействие живых и неживых аспектов находит равновесие в широком диапазоне параметров, поддерживая климат планеты.

Раздел 5

Выполнение лабораторной работы

Реализация на Agents.jl

Создаем файл `scr/daisyworld.jl`, в котором определим тип агента и функции шага модели (рис. 1).



```
1 using Agents
2 import StatsBase
3 using Random
4 @agent struct Daisy(GridAgent{2})
5   breed::Symbol
6   age::Int
7   albedo::Float64 # 0-1 fraction
8 end
9 function update_surface_temperature!(pos, model)
10   absorbed_luminosity = if isempty(pos, model) ? 0 no daisy
11     (1 - model.surface_albedo) * model.solar_luminosity
12   else
13     daisy = model[id_in_position(pos, model)]
14     (1 - daisy.albedo) * model.solar_luminosity
15   end
16   local_heating = absorbed_luminosity > 0 ? 72 * log(absorbed_luminosity) + 80 : 80
17   model.temperature[pos...] = (model.temperature[pos...] + local_heating) / 2
18 end
19 function diffuse_temperature!(pos, model)
20   ratio = model.ratio # diffusion ratio
21   npos = nearby_positions(pos, model)
22   model.temperature[pos...] =
23     (1 - ratio) * model.temperature[pos...] +
24     sum(model.temperature[p...] for p in npos) * 0.125 * ratio
25 end
26 function propagate!(pos, model)
27   isempty(pos, model) && return
28   daisy = model[id_in_position(pos, model)]
29   temperature = model.temperature[pos...]
30   seed_threshold = (0.185 * temperature - 0.0032 * temperature^2) - 0.6443
31   if rand(ABRNG(model)) < seed_threshold
32     empty_near_pos = random_nearby_position(pos, model, 1, npos -> isempty(npos, model))
33     if !isnothing(empty_near_pos)
34       add_agent!(empty_near_pos, model, daisy.breed, 0, daisy.albedo)
35     end
36   end
37 end
```

Рисунок 1: `scr/daisyworld`

Создаем базовую визуализацию. Построим тепловую карту поверхности модели.

```
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab03\project> julia --project=. scripts/daisyworld.jl
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab03\project>
```

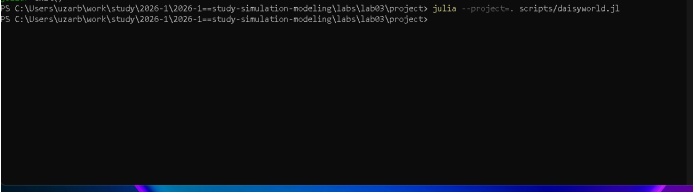
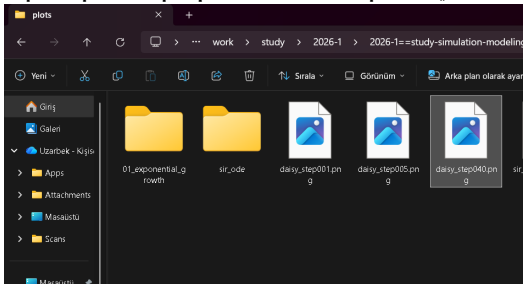


Рисунок 2: scripts/daisyworld

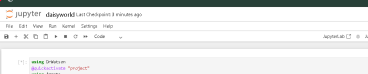
Результаты базовой визуализации (1)

Проверяем графики в папке `plots` ().



Результаты базовой визуализации (2)

Продолжение проверки графиков.



The screenshot shows a JupyterLab environment. At the top, there's a browser address bar with the URL `localhost:8888/lab?token=eyJ0eXciOiJ1cy5udC01L2020-11-03T00:00:00.000Z`. Below the browser bar is a JupyterLab header with a logo and the text "jupyterlab | Last Checkpoint: 3 minutes ago". The main area contains a Jupyter Notebook with the following code:

```

import keras
from tensorflow.keras import layers
import numpy as np

# Load data
data_dir = 'data'
train_data = np.load('train_data.npy')
test_data = np.load('test_data.npy')

# Define model
model = keras.Sequential([
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train model
model.fit(train_data, train_data['target'], validation_data=(test_data, test_data['target']),
          epochs=10, batch_size=32)

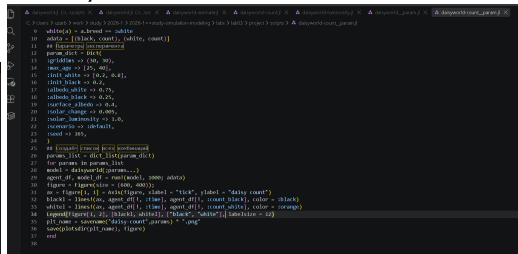
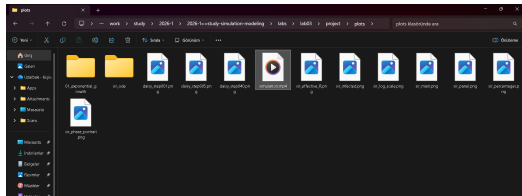
```

The code defines a simple neural network with two hidden layers and a softmax output layer. It uses the Adam optimizer and categorical crossentropy loss. The model is trained for 10 epochs with a batch size of 32. The training data and target labels are loaded from `train_data.npy` and `train_data['target']` respectively. The test data and target labels are loaded from `test_data.npy` and `test_data['target']` respectively.

```
PS C:\Users\azarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab03\project> julia --project=. scripts/daisyworld-animate.jl
```

Производные файлы и Jupyter

Создаем производные файлы из скрипта и запускаем notebook.



Создаю файл `daisyworld-animate` и проверяю создание файла анимации `simulation.mp4`.

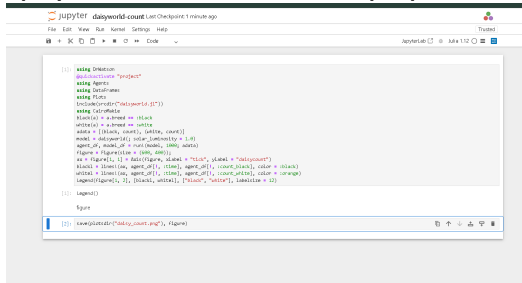
```
PS C:\Users\user\Work\Study\2024-1\2024-1\2024-1\study-simulation-modeling\labs\lab09\projects\julia --project= scripts\daisyworld-count.jl
```

```
C:\Users\user\Work\Study\2024-1\2024-1\2024-1\study-simulation-modeling\labs\lab09\projects>julia --project= scripts\daisyworld-count.jl
generate ns: scripts\daisyworld-count.jl
info: generating plot script file from 'C:\Users\user\Work\Study\2024-1\2024-1\2024-1\study-simulation-modeling\labs\lab09\projects\scripts\daisyworld-count.jl'
info: writing result to 'C:\Users\user\Work\Study\2024-1\2024-1\2024-1\study-simulation-modeling\labs\lab09\projects\scripts\daisyworld-count.jl'
info: generating notebook page from 'C:\Users\user\Work\Study\2024-1\2024-1\2024-1\study-simulation-modeling\labs\lab09\projects\scripts\daisyworld-count.jl'
info: writing result to 'C:\Users\user\Work\Study\2024-1\2024-1\2024-1\study-simulation-modeling\labs\lab09\projects\notebooks\daisyworld-count\julia\daisyworld-count.qmd'
info: generating notebook from 'C:\Users\user\Work\Study\2024-1\2024-1\2024-1\study-simulation-modeling\labs\lab09\projects\scripts\daisyworld-count.jl'
info: writing result to 'C:\Users\user\Work\Study\2024-1\2024-1\2024-1\study-simulation-modeling\labs\lab09\projects\notebooks\daisyworld-count\daisyworld-count.ipynb'
info: notebook: C:\Users\user\Work\Study\2024-1\2024-1\2024-1\study-simulation-modeling\labs\lab09\projects\notebooks\daisyworld-count\daisyworld-count.ipynb

julia> Run below code.
PS C:\Users\user\Work\Study\2024-1\2024-1\2024-1\study-simulation-modeling\labs\lab09\projects
```

Динамика числа маргариток

График изменения числа маргариток в зависимости от модельного времени .



```
[1]: using OrModel
@activate "project"
using Agents
using DataFrames
using Plots
include("daisyworld.jl")
using Interactive
black(a) = a.breed == black
white(a) = a.breed == white
adata = [[black, count, latitude, count]]
model = daisyworld(; solar_luminosity = 1.0)
agent_of, model_of = run(model, 1000, adata)
figure = Figure{Time} = (0.0, 1000)
ax = Figure{1, 1} = Axis{Figure, ylabel = "daisycount"}
blackl = lines!(ax, agent_of[1, :time], agent_of[1, :count_black], color = black)
whitel = lines!(ax, agent_of[1, :time], agent_of[1, :count_white], color = orange)
legend!(figure[1, 2], [blackl, whitel], ["black", "white"], labelsize = 12)

[2]: legend()
figure

[3]: save(plotdir("daisy_count.png"), figure)
```

```
Готово! Все файлы созданы.
PS C:\Users\lazarb\work\study\2026-1\2026-1\astudy-simulation-modeling\labs\lab03\project> julia --project, scripts/daisyworld-luminosity.jl
PS C:\Users\lazarb\work\study\2026-1\2026-1\astudy-simulation-modeling\labs\lab03\project>
```

Файлы динамики маргариток

Генерирую производные файлы и запускаю notebook.

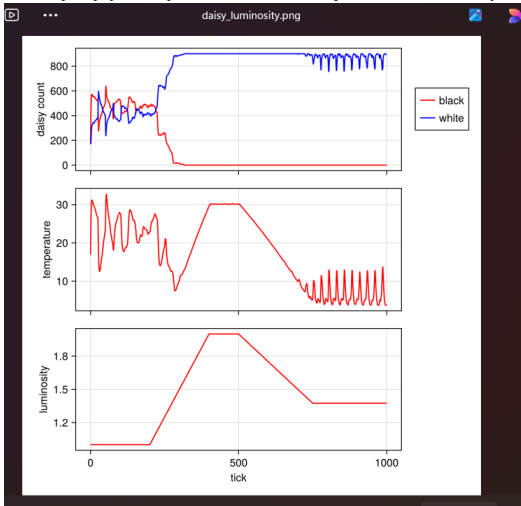
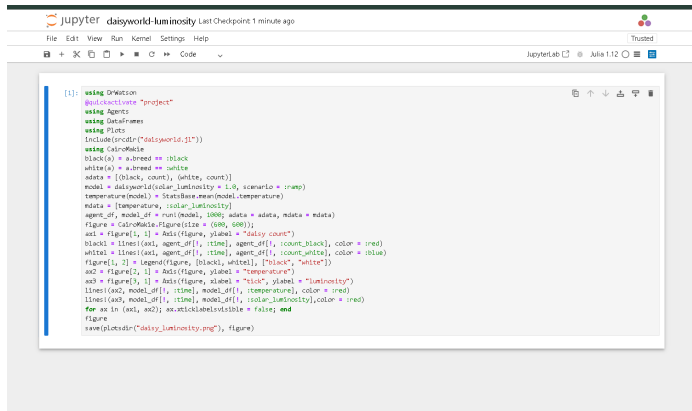
[illegible]

График динамики маргариток

Итоговый график изменения числа маргариток .



```
[1]: using Orthon
@quickactivate "project"
using Agents
using DataFrames
using Plots
include(joinpath("daisyworld.jl"))
using CairoMakie

black(a) = a.breed == :black
white(a) = a.breed == :white
adata = [(black, count), (white, count)]
model = daisyworld(solar_luminosity = 1.0, scenario = :ramp)
temperature(model) = StatsBase.mean(model.temperature)
adata = [temperature, :solar_luminosity]
agent_df, model_df = run!(model, 1000; adata = adata, mdata = mdata)
figure = CairoMakie.Figure(size = (600, 600));
ax1 = figure[1, 1] = Axis(figure, ylabel = "daisy count")
black1 = lines(ax1, agent_df[:, :time], agent_df[:, :count_black], color = :red)
white1 = lines(ax1, agent_df[:, :time], agent_df[:, :count_white], color = :blue)
figure[1, 2] = legend(figure, [black1, white1], ["black", "white"])
ax2 = figure[2, 1] = Axis(figure, ylabel = "temperature")
ax3 = figure[3, 1] = Axis(figure, ylabel = "luminosity")
lines(ax2, model_df[:, :time], model_df[:, :temperature], color = :red)
lines(ax3, model_df[:, :time], model_df[:, :solar_luminosity], color = :red)
for ax in (ax1, ax2); ax.ticklabelsvisible = false; end
figure
save(plotdir("daisy_luminosity.png"), figure)
```

Рисунок 3: plots

Динамика модели (комплексная)

Комплексный график изменения числа маргариток, температуры и альбедо.

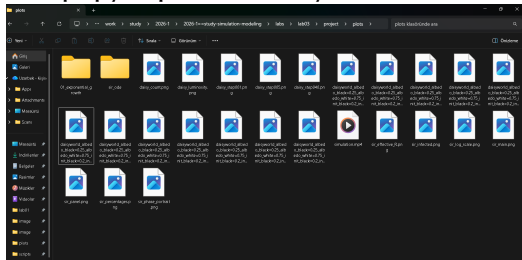
```
C:\Users\cscab> cd c:\work &> cd j2026-1 > 2026-1 --study-simulation-modeling > late > idk03 > project > scripts > daisyworld_param.j
```

```
1 using DelimitedFile
2 #quickactivate "project"
3 using Agents
4 using DataFrames
5 using Plots
6 using CairoMakie
7 include(joinpath("daisyworld.jl"))
8 # @parameters harnegpercent
9 param_dict = Dict{
10     :grid_dims => (30, 30),
11     :max_age => (25, 40),
12     :init_white => [0.7, 0.0],
13     :init_black => 0.2,
14     :albedo_white => 0.75,
15     :albedo_black => 0.25,
16     :surface_albedo => 0.4,
17     :solar_change => 0.005,
18     :solar_luminosity => 1.0,
19     :seedcolor => :default,
20     :seed => 105,
21 }
22 # Equations: luminance, biomass, temperature
23 params_list = dict_list(param_dict)
24 for params in params_list
25     model = DaisyWorld(params...)
26     daisy_color(:xmasidsp) = albreud
27     plot_hags = (
28         agent_color=:daisy_color,
29         agent_size = 20,
30         agent_marker = "o",
31         heatmap = :temperature,
32         heat_hags = (:colorrange = (-20, 60)),
33     )
34     plt1, _ = abaplot(model); plot_hags...))
35     step!(model, 5)
36     plt2, _ = abaplot(model); heatmap = model.temperature, plot_hags...))
37     step!(model, 40)
38     plt3, _ = abaplot(model); heatmap = model.biomass, plot_hags =
```

```
PS C:\Users\uzarb\work\study\2026-2\2026-1=study-simulation-modeling\labs\lab03\projects> julia --project=. scripts\daiyworld_param.jl
```

Производные файлы динамики

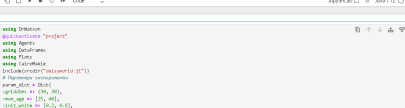
Генерирую файлы и запускаю notebook .



```
PS C:\Users\user\work\study\2020-1\2020-1\study-simulation-modeling\labs\lab09\project> .\scripts\generate_scripts\disgworld_param.pl
Compiling script: scripts\disgworld_param.pl
Info: generating plain script file from 'C:\Users\user\work\study\2020-1\2020-1\study-simulation-modeling\labs\lab09\project\scripts\disgworld_param.pl'
Info: writing result to 'C:\Users\user\work\study\2020-1\2020-1\study-simulation-modeling\labs\lab09\project\scripts\disgworld_param.pl'
Info: generating markdown page from 'C:\Users\user\work\study\2020-1\2020-1\study-simulation-modeling\labs\lab09\project\scripts\disgworld_param.pl'
Info: writing result to 'C:\Users\user\work\study\2020-1\2020-1\study-simulation-modeling\labs\lab09\project\notebooks\disgworld_param.ipynb'
Info: generating notebook from 'C:\Users\user\work\study\2020-1\2020-1\study-simulation-modeling\labs\lab09\project\scripts\disgworld_param.pl'
Info: writing result to 'C:\Users\user\work\study\2020-1\2020-1\study-simulation-modeling\labs\lab09\project\notebooks\disgworld_param.ipynb'
PS C:\Users\user\work\study\2020-1\2020-1\study-simulation-modeling\labs\lab09\project>
```

Базовая визуализация (с параметрами)

Создаю файл `daisy_param` и запускаю его.



The screenshot shows a Jupyter Notebook interface. At the top, the Jupyter logo and the text "Jupyter" are visible. Below the logo, the text "day1world_2" and "Last checkpoint 2 minutes ago" are displayed. The interface includes a menu bar with options: File, Edit, View, Run, Kernel, Settings, Help. Below the menu bar, there are icons for saving, running, and other actions. The main area of the notebook is a code cell containing the following Python code:

```
[4]: using Delimited
      @dict2dict "project"
using Agents
using Delimited
using Plots
using CairoMakie
include("src"/"day1world.jl")
# Parameters
param_dict = Dict{
    :q1_init => (20, 80),
    :max_age => (25, 40),
    :init_white => (0.2, 0.8),
    :init_black => 0.2,
    :init_white => 0.75,
    :init_black => 0.25,
    :surface_init => 0.4,
    :color_change => 0.005,
    :color_likelihood => 1.0,
    :sigma => :default,
    :seed => 101,
}
# Create source from world
param_list = Dict{List{param_list}
for param in param_list
    model = day1world(param...)
    day1color(a::Day1) = subread
    plotwings = (
        agent_color::Day1color,
        agent_size = 20,
        agent_shape = :circle,
        heatmap = :temperature,
        heatmapwings = (colorrange = (-20, 60)),
    )
    plot!, _ = plot!(model; plotwings...)
    step!(model, 5)
    plot!, _ = plot!(model; heatmap = :model.temperature, plotwings...)
    step!(model, 40)
```

[illegible]

Результаты с параметрами

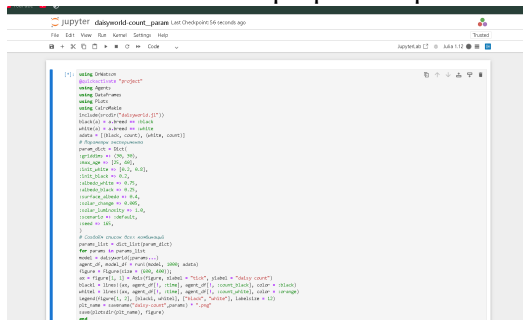
Проверяем создание файлов, генерируем производные и запускаем notebook .

```
Terminal: Run python command.  
P5 C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project julia --project=. scripts\daisyworld-count_param.jl  
P5 C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project
```

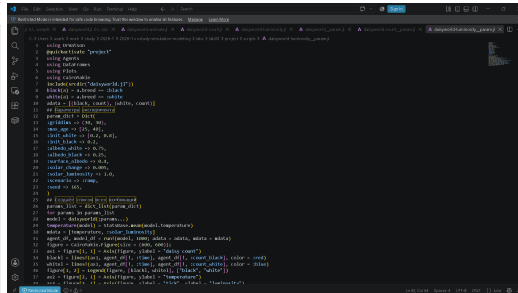
```
P5 C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project julia --project=. scripts\single.jl scripts\daisyworld-count_param.jl  
Response: ex: script\daisyworld-count_param.jl  
Info: generating scala script file from "C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project\scripts\daisyworld-count_param.jl"  
Info: writing result to "C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project\scripts\daisyworld-count_param\daisyworld-count_param.jl"  
P5  
P5 Meta: export: C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project\scripts\daisyworld-count_param\daisyworld-count_param.jl  
Info: generating markdown page from "C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project\scripts\daisyworld-count_param.jl"  
Info: writing result to "C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project\scripts\daisyworld-count_param\daisyworld-count_param.md"  
P5  
P5 Quarto: C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project\notebook\daisyworld-count_param\daisyworld-count_param.qmd  
Info: generating notebook from "C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project\scripts\daisyworld-count_param.jl"  
Info: writing result to "C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project\notebooks\daisyworld-count_param\daisyworld-count_param.ipynb"  
P5  
P5 Notebook: C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project\notebooks\daisyworld-count_param\daisyworld-count_param.ipynb  
Terminal: Run python command.  
P5 C:\Users\uzarb\work\study\2026-1\2026-1--study-simulation-modeling\labs\lab03\project
```

Динамика числа маргариток (параметры)

Изменение числа маргариток с разными параметрами .



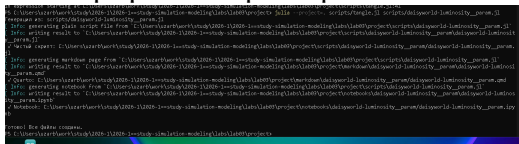
```
[1]: using Orthon
@add_options "project"
using Agents
using DataFrames
using Plots
using Cairo
include(joinpath("daisyworld.jl"))
black(a) = a.breed == black
white(a) = a.breed == white
adata = [black, count], (white, count))
# Parameters
param_list = Dict{
    :grid_size => (50, 50),
    :max_age => 10, :diff,
    :init_black => {0.5, 0.5},
    :init_black => 0.5,
    :init_black => 0.5,
    :init_black => 0.5,
    :surface_albedo => 0.4,
    :color_change => 0.001,
    :color_similarity => 1.0,
    :count => :default,
    :seed => 10,
}
# Create the environment
param_list = Dict{String{Symbol} => Any}()
for param in param_list
    model = daisyworld(param...)
    agent_df, model_df = run(model, 5000, adata)
    figure = Figure(size = (800, 400))
    ax = Figure[1, 1] = Axis{Figure, Symbol}("time", :count_black, color = black)
    black = lines!(ax, agent_df[1, :time], agent_df[1, :count_black], color = black)
    white = lines!(ax, agent_df[1, :time], agent_df[1, :count_white], color = :orange)
    legend!(figure[1, 1], [black, white], ["black", "white"], labelsize = 12)
    plot_name = savefig(joinpath("daisy-count-param"), "daisy-count-param.png")
end
```



```
1 using Orthon
2 @add_options "project"
3 using Agents
4 using DataFrames
5 using Plots
6 using Cairo
7 include(joinpath("daisyworld.jl"))
8 black(a) = a.breed == black
9 white(a) = a.breed == white
10 adata = [black, count], (white, count))
11 # Parameters
12 param_list = Dict{
13     :grid_size => (50, 50),
14     :max_age => 10, :diff,
15     :init_black => {0.5, 0.5},
16     :init_black => 0.5,
17     :init_black => 0.5,
18     :init_black => 0.5,
19     :surface_albedo => 0.4,
20     :color_change => 0.001,
21     :color_similarity => 1.0,
22     :count => :default,
23     :seed => 10,
24 }
25 # Create the environment
26 param_list = Dict{String{Symbol} => Any}()
27 for param in param_list
28     model = daisyworld(param...)
29     figure = Figure(size = (800, 400))
30     ax = Figure[1, 1] = Axis{Figure, Symbol}("time", :count_black, color = black)
31     black = lines!(ax, agent_df[1, :time], agent_df[1, :count_black], color = black)
32     white = lines!(ax, agent_df[1, :time], agent_df[1, :count_white], color = :orange)
33     legend!(figure[1, 1], [black, white], ["black", "white"], labelsize = 12)
34     plot_name = savefig(joinpath("daisy-count-param"), "daisy-count-param.png")
35 end
```

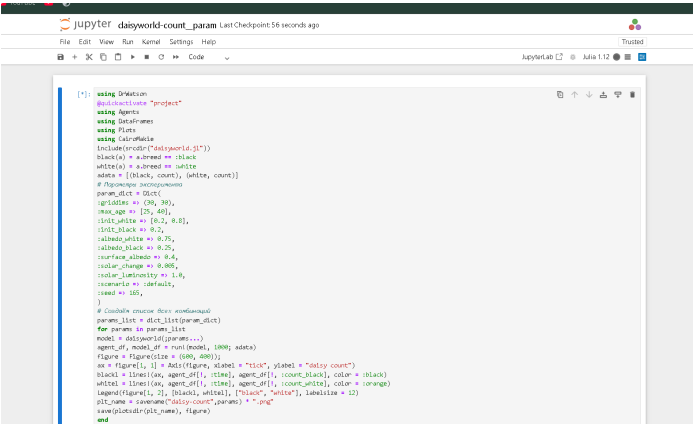
Динамика модели с параметрами

Запускаю файлы и notebook для комплексного графика с параметрами .



Завершение работы

Запускаю итоговый notebook daisyworld-luminosity_param.



```
[*]: using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots
using CairoMakie

include("rodr("daisyworld.jl")")
black(a) = a.breed == :black
white(a) = a.breed == :white
adata = [(black, count), (white, count)]

# Parameters
param_dict = Dict{
    :gridsize => (30, 30),
    :max_age => [25, 40],
    :init_white => [0.2, 0.8],
    :init_black => 0.2,
    :albedo_black => 0.75,
    :albedo_white => 0.25,
    :surface_albedo => 0.4,
    :solar_charge => 0.865,
    :solar_luminosity => 1.0,
    :scenario => :default,
    :seed => 105,
}

# Create the model
param_list = dict_list(param_dict)
for param in param_list
    model = daisyworld(param...)
    agent_df, model_df = run(model, 1000; adata)
    figure = Figure(size = (600, 400))
    ax = Figure[1, 1] Axis{Figure, xlabel = "tick", ylabel = "daisy count"}
    black = lines(ax, agent_df[:, :time], agent_df[:, :count_black], color = :black)
    white = lines(ax, agent_df[:, :time], agent_df[:, :count_white], color = :orange)
    legend{Figure[1, 2], [black, white], ["black", "white"], labelsize = 12}
    plt_name = save_name("daisy-count", param) * ".png"
    save(plotsdir(plt_name), figure)
end
```

Рисунок 4: notebook

Раздел 6

Выводы

В ходе выполнения лабораторной работы был освоен агентный подход к имитационному моделированию на примере модели Daisyworld с использованием библиотеки Agents.jl. (Примечание: Полные листинги кода интегрированы в основной отчёт).